

Project Overview: 2D Convolution Hardware Project

This project specification is contained in five documents. This document contains the overview of the project. (You should start here.) Please see other accompanying PDFs for detailed specifications and tasks for each of the four parts (“Part 1” through “Part 4”) of the project.

1. Introduction

In this project, you will design, implement, simulate, and synthesize a hardware system for performing 2D Convolution. The project consists of four parts, a Final Report, and a live demo; the deadlines and point values are given in Section 2.

We will run additional simulations on the code you turn in, so it is very important to:

1. Make sure your designs simulate correctly using QuestaSim *on the lab computers*.
2. Carefully organize your code as specified in this document.
3. Make sure the names and behavior of all signals match this specification *exactly*.
4. Carefully label and document your code.

Your project will be evaluated on the correctness and efficiency of your design, the quality of your report, and your answers to questions in the report and demo.

You may work alone or with one partner (see Section 1.2). **You may not share code with others (except your partner). This means you may not allow others to see your code, nor may you read others’ code (for this or related projects, including student work from prior years). All code will be run through an automatic code comparison tool. Plagiarism will result in a score of zero on the assignment for all involved parties.**

Please see the project’s generative AI policy (summarized in Section 2.2; full policy in Appendix A). Any AI usage outside of those guidelines will be considered academic dishonesty. If you have questions as to what is acceptable, please come to office hours or send email to ask for clarification.

If you have general questions about the project, please post them to the course board on Piazza.

1.1. File Organization

As you work on the project, please carefully organize your files. Use a separate sub-directory for each of the four parts (called `part1/`, `part2/`, `part3/`, and `part4/`). Then be sure to name your files and modules as specified in the description below. Make sure all your files are stored inside of a private work directory like the `ese507work` directory you made in the HW2 Tool Tutorial.

1.2. Partner

You may work alone or in a team of two students for this project. If you choose to work with a partner, it is important that both partners contribute fully to all phases of the project. Your report will require you to describe each partner’s contribution to the project. Unequal contributions may be reflected in scoring.

If you are choosing to work with a partner, by Wednesday 9/23 at 11:59pm you must:

- Send an email to peter.milder@stonybrook.edu with the subject “ESE 507 Project Partner Signup”
- Send the email from your @stonybrook.edu email address
- In the body of the email, write both your name and your partner’s name. (You do not need to include anything else in the email.)
- CC your partner on the email (using your partner’s @stonybrook.edu email address)

After this, you are committed to working with this partner on this project for the entire semester.

1.3. Getting Started

As you can see, this project is large and complex. This document provides a high-level overview and some important background information. Then, the accompanying documents give the specification and tasks for each of the four parts of the project. Begin by carefully reading this overview, and then you should spend some time briefly familiarizing yourself with Parts 1–4. Then when you are ready to start working, refer closely to the Part 1 document.

2. Submission Deadlines and Policies

This project is structured to have four parts and one Final Report, which will be submitted at the following deadlines.

- **October 26:**
 - Part 1: Multiply-Accumulate Unit [10 points]
 - Part 2: Input Memory Module [20 points]
- **December 7:**
 - Part 3: 2D Convolution [20 points]
 - Part 4: Performance Optimization [15 points]
 - Final Report [25 points]
- **Week of December 7:**
 - Live demos [10 points]

Important: This schedule is set up to help ease you into the project. The schedule gives you “extra” time at the beginning to help you get started. **However, you should be aware that the project gets more difficult as it progresses.** I highly recommend that you attempt to work *ahead* of the October 26th deadline, so you have more time to work on the more challenging portions of the project.

We will schedule **live demos** during the week of December 7th. During this time, you (and your partner if you have one) will schedule a session to review your project and code. During this demo, you will be asked to explain portions of your code and the design decisions behind them. Demo points will be assessed on your ability to answer these questions. During the semester we will set up a procedure to schedule your demo time.

The Final Report will require significant time and effort, and it is worth a large portion of the score. **You should work on the report as you work on the project throughout the semester.** (In other words, after you finish Part 1, you should write the corresponding portion of the report, even though the report isn’t due until the end.)

All submissions are due on Brightspace at 11:59pm on the listed day.

2.1. Project Late Policy

- Each of the parts can be submitted up to five days late at a penalty of 10% per day or part of a day. (That is, 1 hour late has the same penalty as 23 hours.)
- No submissions will be accepted more than five days late. (This also includes final submission of Part 4 and the Final Report. No work will be accepted after December 12.)
- If you are submitting late, please upload your work to the Brightspace hand-in and then email Professor Milder. (You only need to send email for late submissions.)
- Only upload your *final* submission for a deadline. In other words, once you make a submission, I will not accept a late submission from you for the same deadline. (For example, if you submit Part 2 on Monday, I won't consider a late re-submission of Part 2 uploaded on Tuesday.)

2.2. Generative AI Policy

The full generative AI policy for this project is given in Appendix A (at the end of this document). **Please read it carefully before using any AI tools.** The box below summarizes the policy.

Summary (TLDR)

You may use AI as a *learning tool*, but not as your designer, coder, debugger, or writer.

- **Log any AI usage:** submit an AI .txt file with each part describing every use of AI, or stating that you did not use AI. (See Appendix A.1.)
- **Allowed:** asking AI about concepts, SystemVerilog syntax, compile/syntax error messages, and CAD tool usage; proofreading text you have already written yourself. (See full list in Appendix A.2.)
- **Not allowed:** using AI to generate code, make design decisions, debug your design for you, generate or write report answers, and letting AI tools read course-provided files. (See full list in Appendix A.3.)
- **Every line of code and every sentence of your report must be written by you**, and you must be able to explain all of it during your December demo.

This is a summary, not the full policy; please read the full policy in Appendix A at the end of this document.

3. Background

Convolution is a mathematical operation used in many applications including signal processing, image processing, and machine learning. For example, two-dimensional convolutions are the fundamental operation used in convolutional neural networks.

In this project, you will design a hardware system that performs strided two-dimensional convolution with a bias. We will first define and introduce convolution in one dimension in Section 3.1; then Section 3.2 extends it to two-dimensional convolution with a bias term. Finally, Section 3.3 introduces the *stride*, which controls how far the weight matrix moves between output values.

3.1. One-Dimensional Convolution

Let $x[n]$ and $w[n]$ represent one-dimensional input arrays. Let's call $x[n]$ the “input” and $w[n]$ the “weights” or the “kernel.” We can define their convolution $y[n]$ as:

$$y[n] = \sum_{i=-\infty}^{\infty} x[i]w[n-i]$$

You have probably seen an expression of this form (defining the convolution of discrete signals) in a signal processing class. Here we will make some reasonable simplifications. These assumptions are commonly made when using convolution as a building block in machine learning applications such as convolutional neural networks.

- We will assume that the input and weights have finite length.
- We will only calculate the values of y that correspond to locations where x and w fully overlap.
- We will assume that we have already stored the elements of w in the order we need to read them, so we can simplify the expression.

Let's assume that x has length R and w has length K . The output y will then be of length $R_{\text{out}} = R - K + 1$. Now we can re-write the convolution as:

$$y[n] = \sum_{i=0}^{K-1} x[n+i]w[i], \quad n = 0, \dots, R_{\text{out}} - 1$$

Or in pseudocode:

```
for n = 0 to Rout-1:
    y[n] = 0
    for i = 0 to K-1:
        y[n] += x[n+i]*w[i]
```

The outer for loop corresponds to stepping over the values of n . The inner for loop corresponds to the summation.

For example, let $x = [10, -20, 30, 40, -50, 60]$ and let $w = [5, 4, 3]$. Therefore $R = 6$, $K = 3$, and $R_{\text{out}} = 6 - 3 + 1 = 4$. We would calculate our four output values as shown below and visualized in Figure 1.

$$\begin{aligned} y[0] &= x[0]*w[0] + x[1]*w[1] + x[2]*w[2] = 10*5 + -20*4 + 30*3 = 60 \\ y[1] &= x[1]*w[0] + x[2]*w[1] + x[3]*w[2] = -20*5 + 30*4 + 40*3 = 140 \\ y[2] &= x[2]*w[0] + x[3]*w[1] + x[4]*w[2] = 30*5 + 40*4 + -50*3 = 160 \\ y[3] &= x[3]*w[0] + x[4]*w[1] + x[5]*w[2] = 40*5 + -50*4 + 60*3 = 180 \end{aligned}$$

3.2. Two-Dimensional Convolution with Bias

Next, we can extend this definition to two dimensions. Let X represent an $R \times C$ matrix (that is, R rows and C columns) and let W represent a $K \times K$ matrix. We will call X the “input matrix” and W the “weight matrix.”

Their convolution Y will be an $R_{\text{out}} \times C_{\text{out}}$ matrix, where $R_{\text{out}} = R - K + 1$ and $C_{\text{out}} = C - K + 1$. In array notation we could write this operation as:

$$Y[r][c] = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} X[r+i][c+j] \cdot W[i][j], \quad \text{for } r = 0, \dots, R_{\text{out}} - 1$$

and $c = 0, \dots, C_{\text{out}} - 1$

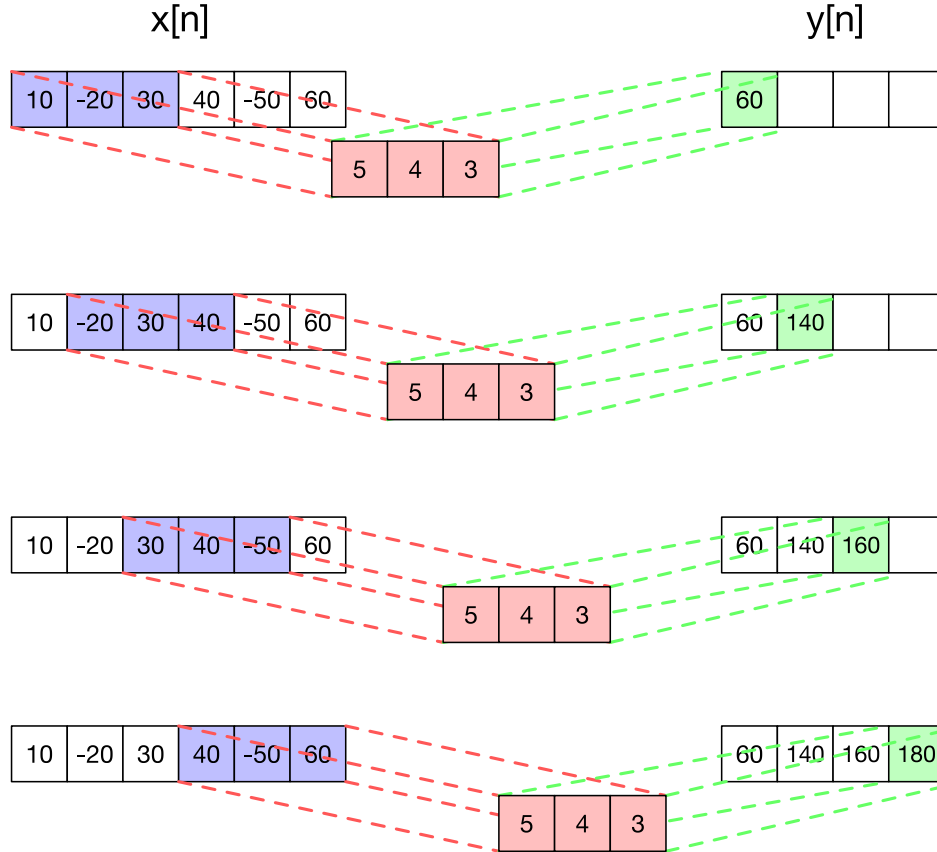


Figure 1: Illustration of the 1D convolution example from Section 3.1.

This looks more complicated, but notice how it is the same structure as 1D convolution, just applied over two dimensions (rows and columns).

We will also introduce a “bias” value B . This is just one extra scalar value added to the convolution, which is commonly used in applications like CNNs. We can now write the operation as:

$$Y[r][c] = B + \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} X[r+i][c+j] \cdot W[i][j]$$

In pseudocode, we could write this operation as:

```

for r = 0 to Rout-1:
  for c = 0 to Cout-1:
    Y[r][c] = B

    for i = 0 to K-1:
      for j = 0 to K-1:
        Y[r][c] += X[r+i][c+j] * W[i][j]

```

Figure 2 illustrates this operation when $B = 0$. (If B were another value, each output would simply have B added to it. For example, if $B = 2$, then the first values would be $6+2=8$ and $3+2=5$.)

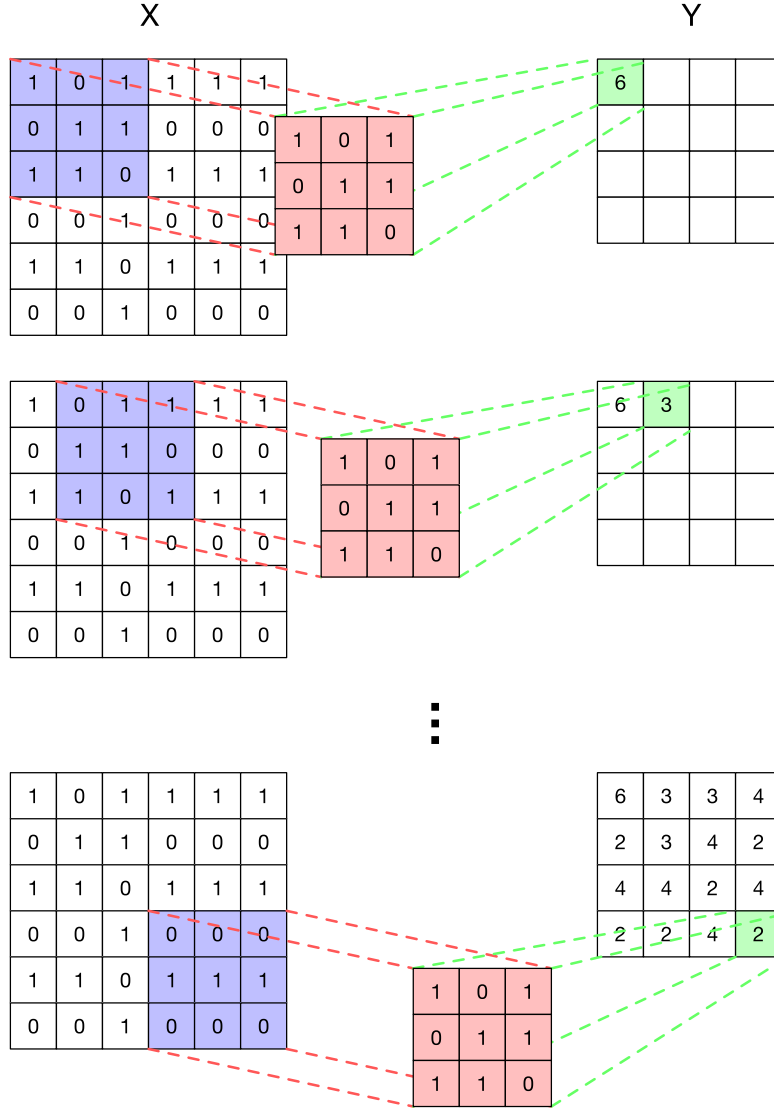


Figure 2: Illustration of 2D convolution with stride 1 (assuming the bias $B=0$).

Computing each of the values in Y requires performing K^2 multiplications and summing up their results. The bias can be handled simply when initializing the value of $Y[r][c]$.

Note that this pseudocode also specifies the order that we compute the values of $Y[r][c]$. That is, first we will compute $Y[0][0]$, then $Y[0][1]$ and up to $Y[0][\text{Cout} - 1]$. Then we move to the left side of the next row and proceed with $Y[1][0]$, then $Y[1][1]$, etc. This is called “row major order,” since we compute the first row of Y entirely, then move on to the second row, and so on. (Your hardware system will also use row-major order for its input and output data; this is explained in more detail in the Part 2 document.)

3.3. Stride

In Section 3.2, the weight matrix moved one position at a time as it passed over the input matrix. (For example, after computing $Y[0][0]$, you would then slide W one column to the right to compute $Y[0][1]$.) It is common in convolutional neural networks to instead move the weight matrix by more than one position at a time. The distance the weight matrix moves is called the *stride*, which we will call S .

With a stride of S , the output value $Y[r][c]$ is computed from the $K \times K$ window of X whose upper-left corner is at row $r \cdot S$ and column $c \cdot S$. So the operation becomes:

```

for r = 0 to Rout-1:
  for c = 0 to Cout-1:
    Y[r][c] = B

    for i = 0 to K-1:
      for j = 0 to K-1:
        Y[r][c] += X[r*S+i][c*S+j] * W[i][j]

```

Because the window now advances S positions at a time, the output is smaller. The output Y is an $R_{\text{out}} \times C_{\text{out}}$ matrix, where:

$$R_{\text{out}} = \left\lfloor \frac{R - K}{S} \right\rfloor + 1$$

$$C_{\text{out}} = \left\lfloor \frac{C - K}{S} \right\rfloor + 1$$

Here $\lfloor \cdot \rfloor$ denotes the “floor operator,” which rounds **down** to the next integer. Notice that any window that would run off the edge of X is simply not computed. (This is why we use the floor operator to round down when computing the output dimensions.) In total, computing all of Y requires $R_{\text{out}} C_{\text{out}} K^2$ multiplications and additions.

Figure 3 repeats the example from above, but now with a stride of 2. In the second convolution step, the window has moved $S = 2$ columns to the right. At the end of the row, it moves down by two rows.

In this project, X , W , B , and S will be inputs to your system, and Y will be an output. R , C , and K will be *parameters* of your hardware system: their values are fixed in your SystemVerilog code when the design is compiled or synthesized, while X , W , B , and S will change while the system runs. For example, if you set parameters $R=6$, $C=7$, and $K=3$, then your hardware system would convolve a 6×7 input matrix X with a 3×3 weight matrix W . If $S = 2$, the output would be a 2×3 matrix, since $R_{\text{out}} = \left\lfloor \frac{6-3}{2} \right\rfloor + 1 = 2$ and $C_{\text{out}} = \left\lfloor \frac{7-3}{2} \right\rfloor + 1 = 3$.

4. High-Level Project Overview

The goal of your project is to build a hardware system for 2D convolution with a bias and a stride. Figure 4 illustrates the top-level module and port specifications of the system. On the left are four signals whose names start with `INPUT_T`. These signals form an AXI-Stream interface that your system will use to receive input data. On the right, there are three signals whose names start with `OUTPUT_T`. These form another AXI-Stream interface your system will use to transmit output data. A specification of the AXI-Stream protocol for the inputs and outputs is provided in Section 5 below. There are also `clk` and `reset` signals. Assume reset is active-high and synchronous. (That is, your system will reset when `reset==1` on a positive clock edge.)

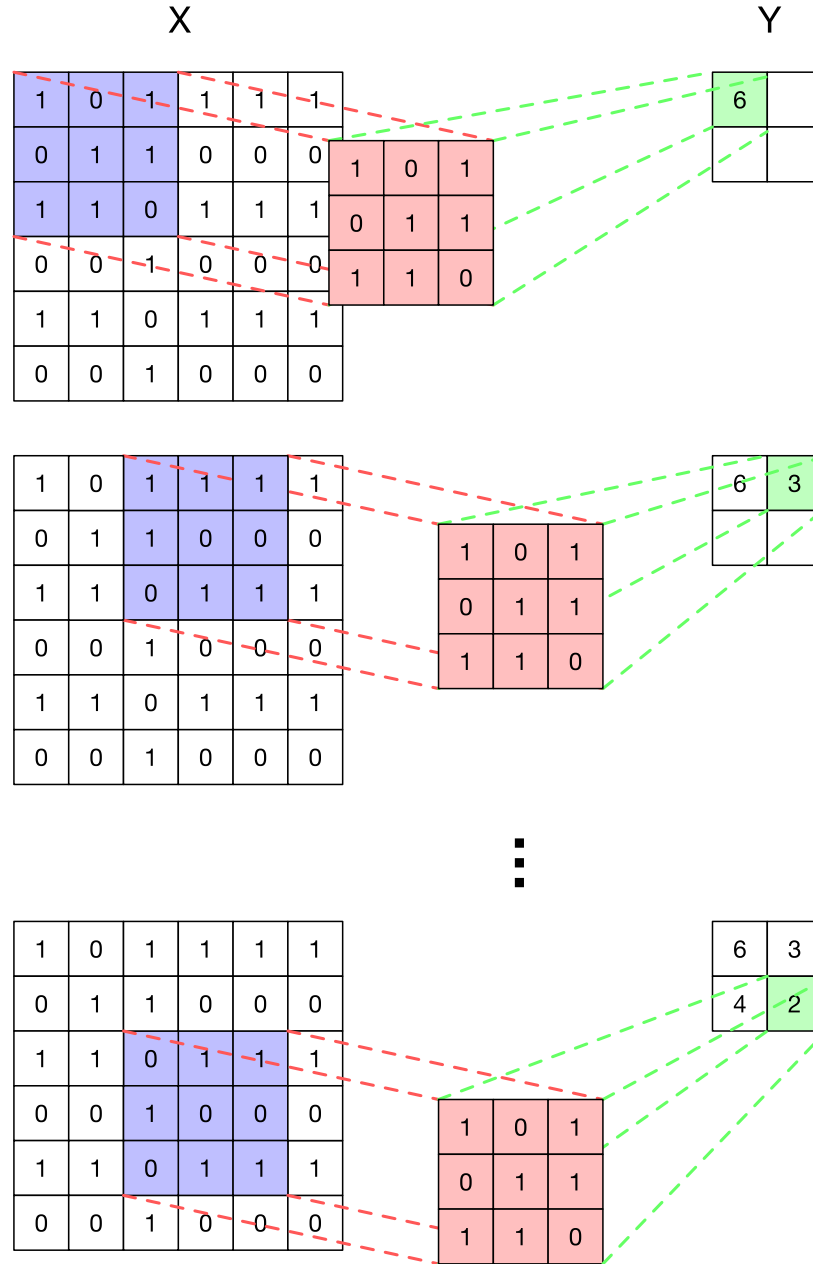


Figure 3: Illustration of 2D convolution with a stride of $S = 2$ (and assuming the bias $B = 0$).

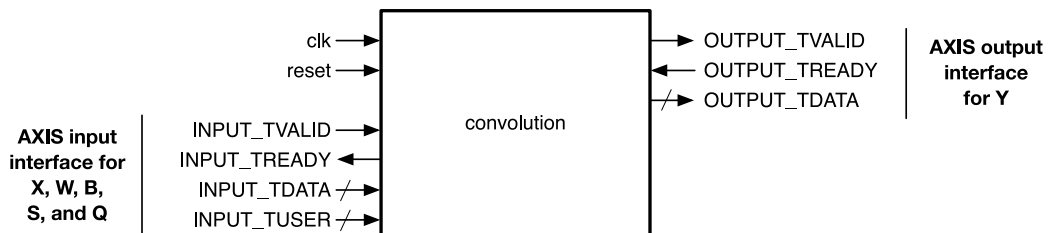


Figure 4: Top-level Design and Port Specifications.

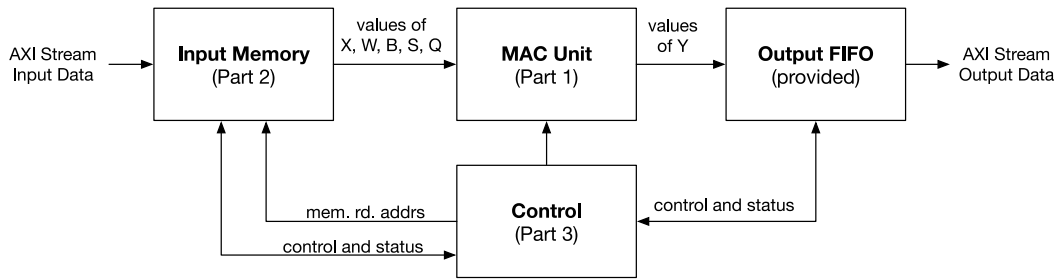


Figure 5: High-Level Block Diagram

4.1. Project Parts

Figure 5 illustrates a high-level block diagram of the system you will construct. Each of the components will be described in more detail in the following sections.

- Your system will take as input a stream of data in AXI-Stream format that represents the X , W , B , S , and Q inputs. (Note that we haven't given you the definition of Q yet; it will be provided in the Part 1 specification.) Your system will perform a convolution (with bias and stride) and produce the result as output. The system's output values will be provided in AXI-Stream format (Section 5). After completing a convolution, your system will accept a new set of inputs to compute. (In other words, your system will keep computing convolutions as long as new inputs are provided.)
- A multiply-accumulate (MAC) unit will be used to perform the individual multiplications and additions needed. A MAC operation computes:

```
f += a*b
```

Notice that this is the fundamental operation used in the convolution pseudocode described above. The MAC unit is **Part 1** of the project, and it is described in the Project Part 1 document.

- Your system will also require input memories to store the input values while the system performs the computation. The Input Memory module, which is **Part 2** of the project, will include memories for X and W , registers to store S , B , and Q , and necessary control logic. You can read more about this in the Project Part 2 document.
- The goal of **Part 3** of the project will be to build a full system for performing two-dimensional convolution. You will do this by integrating the components from Parts 1 and 2 (and a provided Output FIFO) and by designing accompanying control logic that will allow the components to work together. The control logic will be responsible for coordinating the operation of the input memories, MAC unit, and output FIFO. You can read more about this in the Project Part 3 document.
- Lastly, the goal of **Part 4** of the project will be to optimize the speed of the system. Please see the Project Part 4 document for details.

4.2. Parameters

Rather than building hardware for specific matrix dimensions, you will design a *parameterized* system to allow flexibility in the matrix dimensions and in the number of bits used for input and output values. Your top-level system will use the following SystemVerilog parameters.

- R and C : the number of rows and columns of the input matrix X (as defined in Section 3.2 above). Your system must support $R \geq 3$ and $C \geq 3$.
- K : the number of rows and columns of the weight matrix W . Your system must support $K \geq 2$. We also constrain K relative to R and C : $K < C$ and $K < R$. Combined, we would write $2 \leq K < \min(R, C)$.
- **WIDTH**: the number of bits used per number in the input matrices X and W , the bias B , and the output matrix Y . Your system must support $2 \leq \text{WIDTH} < 32$ bits.

There is no defined upper bound on the limits of R , C , and K , but as they get larger, the simulation and synthesis time will grow.

In addition to these top-level parameters, your system's internal modules will define other parameters and `localparams`. Details are provided in the Parts 1–3 description documents.

5. AXI-Stream Input/Output Protocol

Your system will use a slightly simplified version of ARM's AMBA AXI4-Stream protocol. (We will refer to this as AXI-Stream or AXIS for short.)

5.1. Basic AXI-Stream Protocol

AXI-Stream is shown in its simplest form in Figure 6. It is a synchronous protocol (meaning both sides share a common clock) that allows a transmitter¹ module to transfer data to a receiver when both sides “agree.”

The transmitter asserts the **TVALID** signal when it has placed valid data on the **TDATA** signal. The receiver asserts the **TREADY** signal when it is capable of consuming that data. On any positive clock edge, data is transferred if (and only if) both the **TVALID** and the **TREADY** signals are asserted. (No data will ever be transferred unless *both* are asserted.) **TVALID** and **TREADY** are 1-bit signals, while **TDATA** can be multiple bits.

Note that both transmitter and receiver modules must share a common clock. We will call this collection of signals (**TVALID**, **TREADY**, and **TDATA**) an “AXI-Stream interface.” Figure 7 and Table 1 illustrate this functionality and timing. In this example `d[0]`, `d[1]`, etc., represent the data words transmitted.

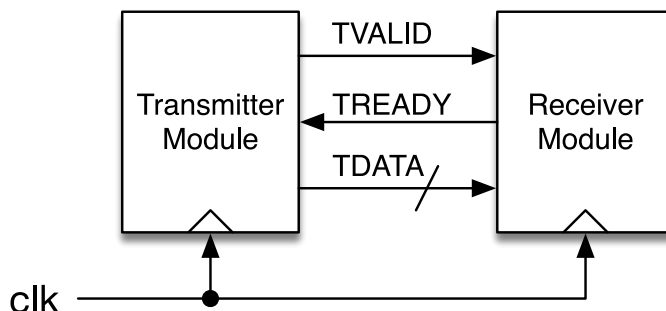


Figure 6: Simplified AXI-Stream protocol signals.

¹In earlier versions of the AXI standard, the transmitter was called a “master,” and the receiver was called a “slave.” In 2021, ARM released a new standard that changed this terminology to “transmitter” and “receiver,” although you will still see the older terms used in some places and CAD tools.

The interaction of the TVALID and TREADY signals is called a *handshake*. Think of asserting TVALID as the transmitter holding out a hand; think of asserting TREADY as the receiver holding out a hand. If both sides hold out their hands, then they shake hands and agree that a data transfer is complete.

In our simplified AXI-Stream protocol, the transmitter is not permitted to wait until TREADY is asserted before asserting TVALID, and the receiver is not permitted to wait until TVALID is asserted before asserting TREADY.² In other words, both the transmitter and the receiver need to decide independently whether to assert their signal. (Then at the positive clock edge, they each *check* to see if the other side has asserted theirs.)

²One small difference between the full AXI-Stream protocol and our simplified version is that in the complete AXI-Stream Protocol, the receiver may choose to wait until TVALID is asserted before asserting TREADY, although we will not allow it in our project. Another difference between our simplified AXI-Stream and the full protocol is that in the full protocol, once the transmitter asserts TVALID, it must keep it asserted until the handshake occurs; we will allow it to be de-asserted at any time.

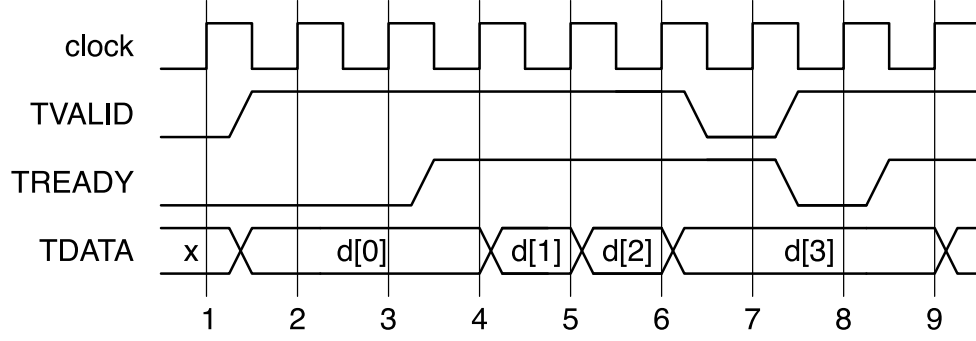


Figure 7: AXI-Stream data transfer timing example.

clock edge #	TVALID	TREADY	Explanation
1	0	0	Neither valid nor ready; nothing is transferred
2	1	0	Transmitter puts data on TDATA and asserts TVALID. However, receiver module hasn't asserted TREADY, so no data is transferred
3	1	0	Receiver module is still not ready (TREADY==0) so no data is transferred
4	1	1	Receiver module is now ready (TREADY==1), so it receives data d[0].
5	1	1	Since d[0] was transferred on the previous clock edge, the transmitter now changes the data to the next word. This word is transferred immediately, since TVALID and TREADY are still asserted.
6	1	1	This has the same logic as clock edge 5. Data word d[2] is transferred.
7	0	1	Now, the transmitter has de-asserted TVALID. The receiver module does not read anything (regardless of what the transmitter has placed onto TDATA).
8	1	0	The transmitter has asserted TVALID but the receiver has de-asserted TREADY. Nothing is transferred here.
9	1	1	Both TVALID and TREADY are asserted, so the receiver reads d[3] from TDATA.

Table 1: Cycle-by-cycle explanation of the data transfers shown in Figure 7.

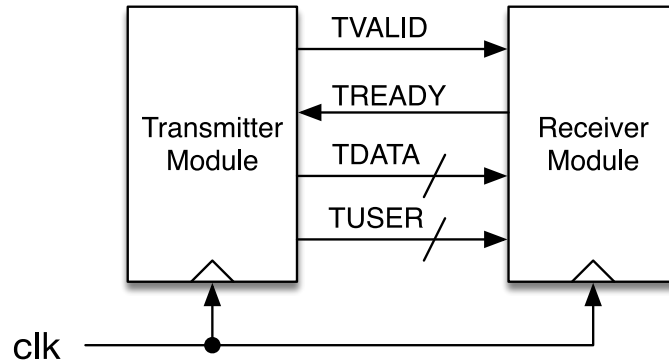


Figure 8: AXI-Stream signals including TUSER.

5.2. Additional AXI-Stream Signal TUSER

In addition to TDATA, TVALID, and TREADY, the AXI-Stream protocol includes several other control signals. In this project, you will use one of them: TUSER, shown in Figure 8.

TUSER is a multi-bit signal that transmits “sideband data” from the transmitter to the receiver. Think of this as extra information that we transmit alongside TDATA. This signal is controlled in exactly the same way as TDATA: any time TREADY and TVALID are 1 on a positive clock edge, then the information on TUSER is also transmitted. The Part 2 description will explain how the TUSER signal is used to transmit the stride S and provide some additional control to your system.

5.3. Output Stream

Your system will utilize TDATA, TREADY, and TVALID for its output. (For output data, you will not use the TUSER signal.) The Output FIFO module will serve as the transmitter. When you simulate, the testbench will be the receiver for this output data. (In a real system, the receiver would be whatever component your system connects to.) So, your Output FIFO will have outputs TDATA and TVALID and input TREADY. The testbench will have inputs TDATA and TVALID and output TREADY.

5.4. Input Stream

Your system’s input interface will use all four signals (including TUSER). Your Input Memory module (Part 2) will serve as the receiver. When you simulate, the testbench will be the transmitter for this input data. For details about how the input data are provided in this stream, please see the Project Part 2 document.

6. Code and Report Submission

This project has five portions: Parts 1–4, where you will submit your designs and synthesis report files, and the Final Report, which is a comprehensive document where you will explain your designs, answer questions, and evaluate your synthesized designs.

Parts 1–4: Each of Parts 1–4 will be submitted as separate submissions on Brightspace, at the deadlines given in Section 2. For each of these parts, you will turn in a single **.zip, .tar, or .tgz** file to Brightspace. *Do not use a different archive format (e.g., .rar). Seriously, we will only accept .zip, .tar, and .tgz archive formats.*

Inside this .zip, .tar, or .tgz file, you will include the following things:

- Your clearly labeled and documented code. (Do not include copies of files that were provided to you like testbenches or simulation scripts unless you modified them.)

- Your synthesis reports for each requested synthesized design.
 - These must be in plaintext format with file names that indicate which design you have synthesized.
 - See the Parts 1–4 documents for the specific designs that you should synthesize.
- A brief plaintext status report called `status.txt`.
 - In this file, state explicitly if your design fully works or not, and explain how you tested it.
 - If the design has any known problems, clearly document them. (You do not need to explain *prior* problems that are now solved. I just want to see details of any *remaining* bugs you are aware of in the submitted design.)
 - Similarly, if you have any uncorrected synthesizability problems, document them here.
 - For example, if it fails simulation or synthesis under certain sets of parameters or situations, explain what is happening to the best of your ability.
 - If your design is perfect, your status report should be very short. Simply state how you tested it, and that you do not observe any problems with your system.
- Your `AI.txt` log as described in Appendix A.1. (If you don't use AI, please include this file with the sentence "I [or We] did not use AI.")

Do not turn in things like CAD tool work directories or intermediate files, copies of the testbenches provided to you, or `gates.v` output files. **Please only submit your code, synthesis reports, and the status report document.**

Your submissions for Parts 1–4 will be evaluated on correctness, efficiency, and the quality and clarity of your code and comments.

Final Report: The Final Report is due on the last day of the project. Your final report should be in **PDF format only** (no Word `.docx` or other formats), and it should be neat, clear, and professional. Your report should be written by you (not AI). Note that the report is worth a large portion of the total project grade, and you are expected to answer each question fully. As noted in Section 2, **you should work on the report as you work on the project throughout the semester**, not just at the end.

Please organize your report neatly. Use headings to separate it into Part 1, Part 2, Part 3, and Part 4. Each part of the project will have a numbered list of questions you should answer. In your report, please use the same numbering to make it easier to find your answers. (In other words, number your answers to match the questions in this assignment.)

Your Final Report will be evaluated on its quality, clarity, completeness, and professionalism.

The Final Report will require you to synthesize designs with given parameters. The synthesis reports documenting these results should be included with the final report. Please organize these files clearly. Your submission must be a `.zip`, `.tar`, or `.tgz` file that includes these synthesis reports and the PDF of your Final Report.

Electronic Hand-In Process: To hand in your project components, go to the course page on Brightspace → Assignments → "Project Part 1/2/3/4" or "Final Report." There you can upload your `.zip`, `.tar`, or `.tgz` file. Only one partner should hand in for the group, but make sure both partners' names are clear in your code and report. All project deadlines are 11:59pm on the given day. (Please see Section 2 above for more information about deadlines and late submissions.)

Uploading Updated Code with Final Report: If you update your Parts 1 or 2 designs while completing the remainder of the project or report, please submit your changed code along with your Final Report. If

you fix major problems from your earlier submissions, I may restore a portion of points lost. (This decision will depend on the severity of the mistake, the quality of improvement, etc.)

For example, imagine that two weeks after you submitted Part 2, you discovered a problem with it and fixed the mistake. In this case, you should re-submit your Part 2 designs and accompanying synthesis reports. If you do this, please include a clearly labeled directory in your Final Report submission .zip, .tar, or .tgz file. Please only include *changed files*. (Don't resubmit work here that you have previously submitted.) Please also mention these changes in your report at the top of the relevant section. Explain clearly what changed and why. It is important that your final submitted code reflect the designs described and quantified in the report.

Creating Archive Files: You can create your .zip, .tar, or .tgz file however you like. On the Linux lab computers, the easiest way is to put copies of everything you're submitting in a single directory (say, handin/) and then, from the directory above it, run:

```
tar cvzf myhandin.tgz handin/
```

To check that it worked, copy myhandin.tgz to another directory and extract it with `tar xvzf myhandin.tgz`.

Appendix A: Generative AI Policy

Generative AI tools such as Claude, ChatGPT, Gemini, Copilot, etc., may only be used in accordance with the following policy. Please read this carefully before using AI tools. Violations of this policy will be considered academic dishonesty.

As we all know, AI tools are now used very frequently in coding and in some aspects of engineering. **When used well**, agents and generative AI have the ability to greatly aid engineers in design, coding, and verification.

In my experience, these tools are very good at *basic tasks*, but on harder challenges they need a strong experienced human hand to guide them to good solutions. This can lead to a dilemma for anyone trying to learn a new skill: **if you over-rely on AI assistance on the easy tasks, you will not develop the skills needed to solve the hard problems when you get to them.** For example, any AI tool would be able to give you a solution to Part 1 of this project, but if you don't design that system yourself and write the code yourself, then you've failed to learn anything, and you've failed to prepare for the harder parts of the project. In the past, when students over-relied on AI in this class, they were left with nonfunctional projects that they didn't understand well enough to fix, and with obvious signs that they did not write their own code or report.

Every line of code in your project and every sentence in your report must be one you have written yourself. You must be able to explain every aspect of your design and line of code. (Don't forget we will have a live demo/design review in December.)

So, our project AI policy is geared to allowing you to use AI as a *learning tool* (if you choose to do so). You may not use AI to make design decisions, write your code, or draft your report. You are also required to document your AI usage as part of the project.

If you have any thoughts or concerns about this policy, or if you find that parts of it are unclear, or if you just have any questions, please come talk about it with me or post on Piazza.

A.1. Required AI Log

If you use AI in any way, then you must keep and submit a log that briefly documents what you used it for. In each of the four parts of your project, keep a plaintext `AI.txt` file that briefly describes each time you used AI and how you used it. If you are working with a partner, include your name with your entries, and your partner's name with their entries. Each entry should say who used the tool, the date, which AI tool you used, and what you used it for. Missing or incomplete AI logs will result in point deductions.

Some examples:

- “[Your name], October 3, I asked Claude to help me understand the meaning of TREADY and TVALID in AXI-Stream”
- “[Your partner's name], October 15, I pasted into Gemini a QuestaSim error message and the line of my code it pointed to; it explained that the error came from an unsized literal.”
- “[Your name], October 16, I asked ChatGPT what the slack column means in the DesignCompiler timing report.”
- “Both partners, October 18, we asked ChatGPT to edit the grammar on question 3 of the report.”

(If you are working without a partner, you can skip the name in each entry.)

If you do not use AI at all on your project or report, then your AI .txt file will consist of a single sentence: “I [or We] did not use AI.” (But you are still required to include this file.)

A.2. Permitted Uses of AI

- **Concepts:** You can use AI to help understand concepts and ideas. For example, “How does AXI-Stream handshaking work?” or “How do I interpret the DesignCompiler report_timing output?” Think of these as questions you could ask in class or office hours. (And of course you can still ask those questions in class or office hours too!)
- **Language reference:** You can ask basic questions about SystemVerilog syntax (or other languages if needed). Think of this as the type of information you would read in a book or reference manual. You can also use AI to understand syntax and compile errors: you can paste an error message from the tools along with the line or two of your code it points to, and ask what the error means. Note the difference between pasting a line or two of code to ask for language help (which is allowed) and pasting larger amounts of code and asking it to debug your design.
- **Tool reference:** Questions about how to use the tools are great. For example, questions about how to better use QuestaSim or Design Compiler, or what to run at the command line to accomplish a given task are fine. However, keep in mind that sometimes LLMs can be confidently **incorrect** about tool syntax and capability.
- **Proofreading:** You can use AI to help with editing your human-written text for grammar and clarity, but be careful that it doesn’t get facts wrong in its suggestions. **You must write your own text yourself before you ask AI to edit it.**

A.3. Prohibited Uses of AI

- **Any tool that writes code into your project files or generates project files:** You may not use AI-generated code or text. This includes content produced by coding agents or copy/pasting from a chat window or using AI-based autocomplete. AI can help you understand the ideas, but you are responsible for writing the code.
- **Asking AI to make design decisions or produce design content:** Whether you use it directly or indirectly, you can’t ask AI to solve design problems for you. For example, this would include designing logic for you, fixing your broken FSM, analyzing your design for the report, or reasoning about optimization methods for Part 4.
- **Debugging your design:** You may not paste your code or significant parts of it into an AI tool and ask it to find your bug or ask why it doesn’t work, or why the simulation failed. (This also includes running a coding agent in your project directory and asking it questions about your design, even if the agent does not modify any files.) This is different from using AI to help with syntax and compile errors described above. The difference is that if QuestaSim or DesignCompiler tell you there’s a syntax error, it’s fine to ask AI about that line and the error message. But after you’ve compiled your code and you’re running simulation, debugging it is your job, not the AI’s. (If answering your question requires the AI to know what the design is *supposed to do*, then it’s not a syntax question.)

This rule is in place because I’ve found that debugging your system, including interpreting simulation waveforms, tracking the bug’s effects, and reasoning about the cause, is one of the most valuable skills students learn from projects like these. (It’s also a skill you will need especially as you get to more

complex designs here or in the future.) If you're stuck on a bug, come talk to me. Helping you with questions like this is one of the most important ways to use office hours.

- **AI-written report answers:** You may not ask the AI to write portions of your report or to figure out the answers for you. The report is where you are asked to show understanding of your design and to analyze its quality. In the past, when students attempted to use AI tools on the report, they submitted answers that were typically very general and not fully responsive to the question; they also were clearly not written by the student. As described above, you can use AI to help improve your writing, but it has to be **your writing**.
- **Retyping or paraphrasing AI content:** For all of the examples above, re-typing the AI's output or paraphrasing its text does not change its acceptability. If you're just retyping AI-generated code or typing "your own" version of it while looking at the AI-generated code, that's not really any different from copy/pasting it. Similarly, if you ask AI to generate an FSM and then you re-draw it on paper, that doesn't mean you designed it.
- **Uploading course-provided files:** You may not allow AI tools to read course-provided project files including these project specification PDFs, testbenches, and code. This includes uploading them in a chat window or NotebookLM/Gemini Notebook, or putting them in a location where a coding agent (like Claude Code or Codex) can access them.